THAI NGUYEN UNIVERSITY OF INFORMATION AND COMMUNICATION TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY



COURSE PROJECT REPORT

TOPIC:

A TEAM PROJECT MANAGEMENT SYSTEM WITH INTEGRATED AI

Supervisor: Hà Thị Thanh

Students: Phạm Văn Huynh (team leader)
Đàm Đức Đôn
Phạm Minh Tú

Class: Computer Engineering K23A

Thai Nguyen, 2026

THAI NGUYEN UNIVERSITY OF INFORMATION AND COMMUNICATION TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY



COURSE PROJECT REPORT

TOPIC:

A TEAM PROJECT MANAGEMENT SYSTEM WITH INTEGRATED AI

Supervisor: Hà Thị Thanh

Students: Phạm Văn Huynh (team leader)
Đàm Đức Đôn
Phạm Minh Tú

Class: Computer Engineering K23A

Thai Nguyen, 2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to the lecturers of the Faculty of Information Technology, Thai Nguyen University of Information and Communication Technology, for the knowledge and guidance they have given us throughout the course. In particular, we thank our supervisor for the advice that shaped this project—including the suggestion to support the Scrum process, which became one of the main features of the system.

We also thank our classmates in Computer Engineering K23A who used the public test instance and reported problems, several of which are described in this report.

Although we have tried our best, the project and this report certainly still have shortcomings. We look forward to receiving comments from the lecturers so that we can improve them.

Thai Nguyen, 2026

The project team

Phạm Văn Huynh Đàm Đức Đôn Phạm Minh Tú

TABLE OF CONTENTS

ACKNOWLEDGEMENTS **i**

LIST OF TABLES **iv**

LIST OF FIGURES **v**

LIST OF ABBREVIATIONS **vi**

INTRODUCTION **1**
0.1. Rationale 1
0.2. Objectives 1
0.3. Scope 1
0.4. Methodology 2
0.5. Team organisation and task allocation 2
0.6. Structure of the report 3

CHAPTER 1: THEORETICAL BACKGROUND **4**
1.1. Project management for small software teams 4
1.1.1. Kanban 4
1.1.2. Scrum 4
1.2. Dependency graphs and the critical path 4
1.2.1. Topological ordering 4
1.2.2. Critical path 5
1.3. Local-first software 5
1.4. Large language models and structured output 5
1.5. Technologies 6
1.6. Documentation as a control system 6

CHAPTER 2: REQUIREMENTS ANALYSIS **7**
2.1. Survey of existing tools 7
2.2. Actors 8
2.3. Functional requirements 8
2.3.1. Scrum workflow 9
2.4. Non-functional requirements 10

CHAPTER 3: SYSTEM DESIGN **11**
3.1. Overall architecture 11
3.2. Data model 12

3.3. API design 12
3.4. Authorisation design 13
3.5. AI cascade 13
3.6. Offline policy and synchronisation 14
3.7. User interface design 14

CHAPTER 4: IMPLEMENTATION AND TESTING **15**
4.1. Development environment 15
4.2. Implemented features 15
4.2.1. Landing page and authentication 15
4.2.2. Projects, members and roles 16
4.2.3. Kanban board and table view 17
4.2.4. Task inspector: discussion, evidence and history 18
4.2.5. Scrum 19
4.2.6. AI tools 22
4.2.7. Dependency graph, notifications and profile 24
4.3. Testing 26
4.3.1. Strategy 26
4.3.2. Results 27
4.3.3. Defects found 27
4.4. Deployment 28

CONCLUSION **29**

REFERENCES **31**

LECTURER'S COMMENTS **32**

LIST OF TABLES

Table 1. Task allocation among team members 2
Table 1.1. Technology stack 6
Table 2.1. Comparison with common tools 7
Table 2.2. Actors of the system 8
Table 2.3. Functional requirement groups 9
Table 2.4. Main non-functional requirements 10
Table 3.1. API overview 13
Table 4.1. Development environment and project size 15
Table 4.2. Test results 27
Table 4.3. Selected defects found and fixed 27

LIST OF FIGURES

Figure 2.1. Main use cases (a PM can do everything a Member can) 8
Figure 2.2. Sprint lifecycle 10
Figure 3.1. Deployment and component architecture 11
Figure 3.2. Entities (10 tables) 12
Figure 3.3. Three-tier AI cascade 14
Figure 4.1. Landing page 16
Figure 4.2. Sign-in dialog 16
Figure 4.3. Project dashboard with per-project roles 17
Figure 4.4. Kanban board 17
Figure 4.5. Table view 18
Figure 4.6. Keyboard shortcuts 18
Figure 4.7. Task inspector with discussion and history 19
Figure 4.8. Scrum panel: sprint lifecycle 20
Figure 4.9. Scrum panel: backlog planning 20
Figure 4.10. Scrum panel: burndown chart 21
Figure 4.11. Scrum panel: velocity 21
Figure 4.12. Scrum panel: AI retrospective 22
Figure 4.13. AI weekly progress summary 23
Figure 4.14. AI meeting minutes 23
Figure 4.15. AI workload analysis and assignment 24
Figure 4.16. AI goal decomposer 24
Figure 4.17. DAG execution order and critical path 25
Figure 4.18. Notification feed 25
Figure 4.19. Profile page 26

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CPM	Critical Path Method
CRUD	Create, Read, Update, Delete
DAG	Directed Acyclic Graph
DEC	Decision record (in the development book, D7)
E2E	End-to-end (testing)
i18n	Internationalisation
JWT	JSON Web Token
LLM	Large Language Model
ORM	Object-Relational Mapping
PM	Project Manager (a per-project role)
REST	Representational State Transfer
RTM	Requirements Traceability Matrix
SPA	Single-Page Application
TLS	Transport Layer Security
UI / UX	User Interface / User Experience
WIP	Work In Progress

INTRODUCTION

0.1. Rationale

Small software teams—student groups in particular—usually manage their work with chat messages, spreadsheets or a general-purpose tool such as Trello or Jira. Each choice has a cost. Chat loses information; spreadsheets know nothing about status, dependencies or deadlines; and the large tools are heavy, mouse-driven, require a constant connection and charge for the features a team actually needs, such as Scrum reports. Meanwhile, two developments make a better tool practical: browsers can now hold a full working copy of a project locally (the *local-first* idea [1]), and large language models can turn unstructured text—meeting notes, a vague goal—into structured, actionable data.

This project, **Koshi**, builds a project management system around three ideas: every action is reachable from the keyboard; the board keeps working when the network does not; and AI is used for the tedious parts of management (summaries, minutes, assignment, task breakdown, sprint retrospectives) while always returning structured, checkable output.

0.2. Objectives

- Build a web application in which users register, create projects and invite members, with roles (Project Manager or Member) assigned *per project* rather than per account.
- Provide a four-column Kanban board and a table view, fully operable from the keyboard, with dependencies between tasks and an automatically computed critical path.
- Support the Scrum process: sprint planning, a product backlog, sprint start and completion, burndown and velocity charts, and an AI-written retrospective.
- Integrate an LLM (Groq, model `openai/gpt-oss-120b`) for five AI features, with a three-tier fallback so that the application never fails when the AI service is unavailable.
- Keep a personal project fully usable offline, while preventing conflicting offline edits in shared projects.
- Deploy a production-configured instance on a public domain for the team to test, and verify the system with automated tests.

0.3. Scope

The system targets small teams (roughly 2–15 people) working on software projects. It is a web application usable on desktop and mobile browsers, in English and Vietnamese. Out of scope are time tracking, billing, Gantt charts, native mobile applications and integration with third-party issue trackers.

0.4. Methodology

The team applied the process the product supports. Work was organised in sprints with a shared backlog; each change was accompanied by tests, and every non-trivial decision was recorded. The project keeps eight living documents (D1 Requirements, D2 Module map, D3 Architecture, D4 API and data contracts, D5 Tests and acceptance criteria, D6 Risks and policies, D7 Development book and decision log, D8 Requirements traceability matrix) in the same repository as the code, so that a change to the code and the documents describing it are reviewed together. Testing followed a *characterisation-first* approach: before changing untested code, a test describing its current behaviour was written; tests themselves were checked by *mutation testing* [2]—deliberately breaking the code and confirming that a test fails.

0.5. Team organisation and task allocation

The team has three members from class Computer Engineering K23A. Table 1 shows how the work was divided.

Table 1. *Task allocation among team members*

Member	**Role**	**Responsibilities**
Phạm Văn Huynh (KTMT K23A)	Team leader Scrum Master	• Project planning, sprint organisation and progress tracking • Requirements analysis and the documentation set D1–D8 • Scrum module: sprint lifecycle, backlog planning, burndown, velocity, AI retrospective • AI integration with Groq and prompt design; language-aware AI answers • Deployment: Docker, MySQL, Cloudflare Tunnel on `koshi.huynhca.com` • Test data, end-to-end smoke tests, demo video, this report

Member	Role	Responsibilities
Phạm Minh Tú (KTMT K23A)	Lead developer (architecture, frontend)	• Overall architecture and technology choices • Vue 3 frontend: Pinia store, Kanban and table views, keyboard engine • Dependency graph, topological sort and critical path • Local-first storage (IndexedDB) and the offline write policy • Landing page, profile page, localisation (English/Vietnamese)
Đàm Đức Đôn (KTMT K23A)	Developer (backend, quality)	• FastAPI backend: authentication, projects, per-project roles and invitations • Comments, @mentions, replies, attachments, notifications, task history • Database schema, Alembic migrations, migration from SQLite to MySQL • Continuous integration (GitHub Actions), backend and frontend test suites • AI fallback logic and key rotation

0.6. Structure of the report

Besides the introduction and conclusion, the report has four chapters. Chapter 1 presents the theoretical background: Kanban, Scrum, dependency graphs and the critical path, local-first software, LLMs and the technologies used. Chapter 2 analyses the requirements. Chapter 3 describes the system design: architecture, data model, API, AI cascade, security and offline policy. Chapter 4 presents the implementation with screenshots, the testing strategy and its results, and the deployment.

CHAPTER 1: THEORETICAL BACKGROUND

This chapter presents the project management methods the system supports, the algorithms behind its dependency features, and the technologies used to build it.

1.1. Project management for small software teams

1.1.1. Kanban

Kanban visualises work as cards moving across columns that represent states, and limits the amount of work in progress so that the team finishes work before starting more [3]. Its strength is continuous flow: there is no fixed iteration, and a blocked item is visible immediately. Koshi uses a fixed four-state workflow:

$$\text{TODO} \rightarrow \text{IN_PROGRESS} \rightarrow \text{BLOCKED} \rightarrow \text{DONE} \rightarrow \text{TODO}.$$

The same cycle is implemented on the client and on the server and must stay identical; a task in BLOCKED carries a reason, so the board explains *why* work has stopped.

1.1.2. Scrum

Scrum is a framework in which a team delivers work in fixed-length iterations called *sprints*, usually one to four weeks long [4]. Its artefacts are the *product backlog* (everything that might be done), the *sprint backlog* (what the team commits to in this sprint) and the increment. Its events are sprint planning, the daily scrum, the sprint review and the *sprint retrospective*, in which the team inspects how it worked and decides what to change. Two measures are standard:

- **Burndown**: the remaining estimated work (story points) per day of a sprint, compared with an ideal straight line from the commitment down to zero.
- **Velocity**: the story points completed per sprint, used to size the next commitment realistically.

Koshi supports Scrum on top of the Kanban board: a task belongs either to the product backlog or to one sprint; story points are the task's complexity (S=1, M=2, L=3, XL=5).

1.2. Dependency graphs and the critical path

1.2.1. Topological ordering

If task A depends on task B, B must be finished first. The tasks and dependencies form a directed graph $G = (V, E)$ with an edge $B \rightarrow A$. When G has no cycle (a *directed acyclic graph*, DAG), a *topological order*—a sequence in which every task appears after all its prerequisites—exists and can be found with Kahn's algorithm [5]: repeatedly take a task with no unfinished prerequisite, output it, and remove its outgoing edges. Koshi breaks ties between ready tasks by priority, then due date, then creation time, so the suggested order puts the most urgent unblocked work first. A cycle (which users can

create by mistake) does not make the algorithm fail: tasks left in a cycle are appended at the end.

1.2.2. Critical path

The Critical Path Method [6] finds the longest chain of dependent activities; any delay on it delays the whole project. Koshi computes the heaviest chain of *unfinished* tasks with memoised depth-first search, where the weight of a task t is

$$w(t) = p(t) \cdot c(t), \qquad p \in \{1,2,5,10\} \text{ (LOW..CRITICAL)}, \quad c \in \{1,3,5,8\} \text{ (S..XL)},$$

and the weight of a chain is the sum of its tasks. Tasks on this path are flagged on the board and in the DAG view. A subtle bug found during the project concerned memoisation across a cycle: a value computed for a walk truncated by a cycle depends on the path taken, so caching it made the result depend on the order of the input array. Such results are now never cached.

1.3. Local-first software

Local-first software keeps the primary copy of the user's data on the user's device and treats the server as a synchronisation peer [1]. The benefits are instant response and offline operation; the main difficulty is merging concurrent edits. Koshi adopts a deliberately conservative policy: the browser writes every change to IndexedDB first and renders it immediately, then synchronises with the server; while offline, a *personal* project (one member) stays editable, whereas a *shared* project becomes read-only, because without a merge algorithm two people editing offline would silently overwrite each other.

1.4. Large language models and structured output

Large language models based on the Transformer architecture [7] can summarise, extract and plan from natural language. Their output is free text, however, and an application cannot act on a paragraph. Koshi therefore asks the model for JSON with a fixed schema (for example `{went_well, to_improve, action_items}`), strips Markdown fences, validates and normalises the result (unknown enum values become a default; dependencies naming a task that does not exist are dropped), and falls back to a deterministic answer when the output is unusable. The model used is `openai/gpt-oss-120b` [8] served by Groq through an OpenAI-compatible API [9].

1.5. Technologies

Table 1.1. *Technology stack*

Layer	**Technology**	**Purpose**
Frontend	Vue 3, TypeScript, Pinia, Vite, Tailwind CSS	Single-page application; Composition API components and a central store [10]
Local storage	IndexedDB (idb-keyval)	Local-first copy of each project's tasks
Backend	Python, FastAPI, Pydantic v2	REST API with typed request/response schemas [11]
Data	SQLAlchemy 2, Alembic, MySQL 8.4	ORM, versioned migrations, production database (SQLite for development)
Security	JWT (HS256), bcrypt	Stateless sessions [12], password hashing
AI	Groq API, Ollama (optional)	Cloud LLM, optional local LLM
Testing	pytest, Vitest, Vue Test Utils, Playwright	Unit, component and browser automation
Delivery	Docker Compose, nginx, GitHub Actions, Cloudflare Tunnel	Containers, reverse proxy, CI, public access without opening ports [13]

1.6. Documentation as a control system

Documentation that describes a system becomes wrong as soon as the code changes without it; this project inherited a specification describing behaviour the code did not have. The team therefore kept eight documents next to the code (D1–D8, listed in the Introduction), with a rule that a code change invalidating a document is incomplete until the document is updated in the same commit. D7 records every significant decision (*DEC-nnn*) and every defect found (*F-nn*) with the reason behind it, and D8 traces each requirement to the code and the tests that verify it. These records are the source of the figures reported in Chapter 4.

CHAPTER 2: REQUIREMENTS ANALYSIS

This chapter analyses existing tools, identifies the actors and states the functional and non-functional requirements. The full register (89 functional and 10 non-functional requirements; the group counts below sum to 89) is kept in document D1; the most important ones are summarised here.

2.1. Survey of existing tools

Table 2.1. *Comparison with common tools*

Criterion	**Trello**	**Jira**	**Spreadsheet**	**Koshi**
Kanban board	Yes	Yes	Manual	Yes
Scrum reports (burndown, velocity)	Add-on	Yes	No	Yes
Task dependencies, critical path	No	Partial	No	Yes (DAG)
Full keyboard control	Limited	Limited	Yes	Yes
Works offline	No	No	Yes	Personal projects
AI summaries, minutes, assignment	Paid	Paid	No	Yes
Learning curve for students	Low	High	Low	Low

The survey shows a gap the project targets: a tool as light as Trello that still offers Scrum reporting, dependency analysis and AI assistance, and that remains usable without a connection.

2.2. Actors

Table 2.2. *Actors of the system*

Actor	Description
Visitor	Not signed in. Sees the landing page (features, demo video, pricing, FAQ) and can register or sign in.
User	Any signed-in account. Has *no* authority by itself: it can create projects, answer invitations and edit its own profile.
Project Manager (PM)	A user holding the PM role *in a given project*. Manages members and roles, plans and runs sprints. The creator of a project becomes its PM.
Member	A user holding the Member role in a project. Creates and updates tasks, comments, uses the AI tools; cannot change roles or sprints.
AI service	External LLM (Groq) or local model (Ollama) that answers structured prompts.

Roles are deliberately per project: the same person can be PM of one project and Member of another. Registration never asks for a role.

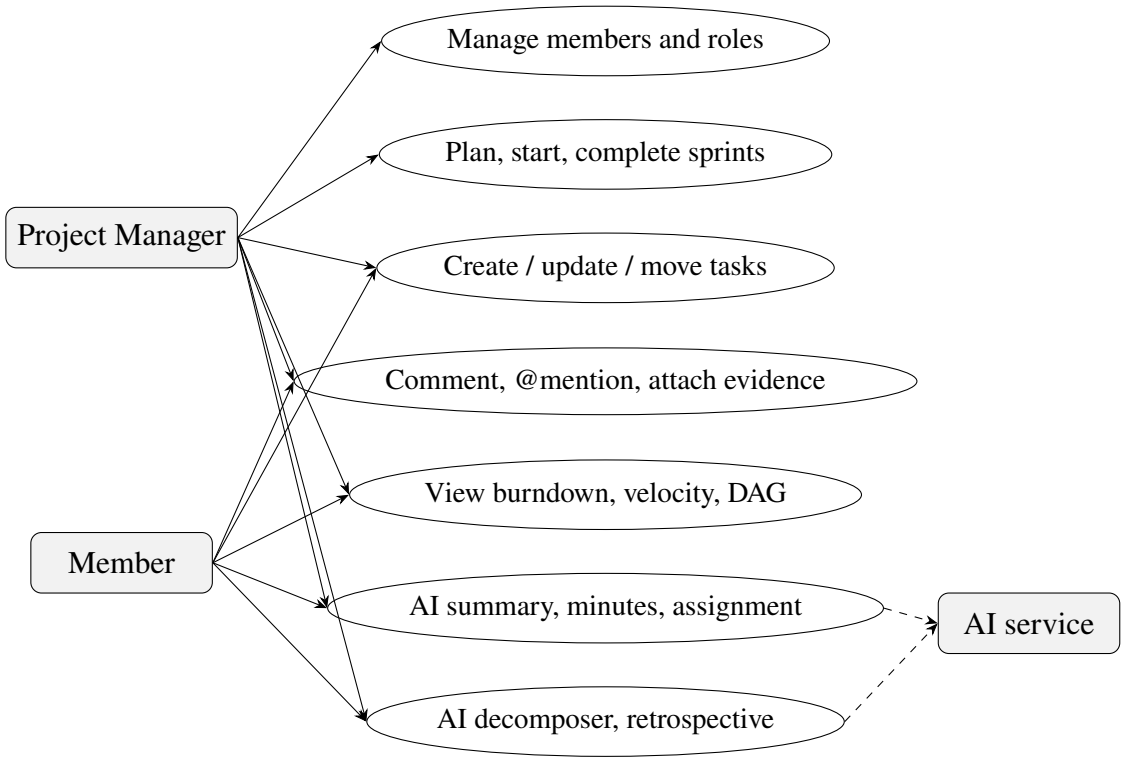


Figure 2.1. *Main use cases (a PM can do everything a Member can)*

2.3. Functional requirements

Table 2.3. *Functional requirement groups*

Group	**Representative requirements**	**Count**
FR-AUTH	Register with email and password (no role); sign in; JWT session; per-project roles; non-members receive 404 for a project they cannot see.	10
FR-PROJ	Personal dashboard; create and delete projects; invite by email (pending until accepted); change roles; profile page with avatar.	11
FR-DOM	Task with title, description, status, priority, complexity, due date, blocking reason, several assignees, dependencies, acceptance criteria; comments with @mentions, one-level replies and attachments; notifications.	10
FR-INT	Kanban and table views; every action on the keyboard ($h/j/k/l$, Space to cycle status, n new, Enter details, / search, 1–4 priority); deadline-first ordering.	14
FR-GRAPH	Topological order, cycle tolerance, critical path highlighted on the board and in a DAG view.	5
FR-PERS	IndexedDB first, server second; personal projects editable offline, shared projects read-only offline.	6
FR-SCRUM	Sprint lifecycle (planned, active, completed); one active sprint per project; commitment recorded at start; unfinished work moved on completion; backlog planning; burndown; velocity; AI retrospective.	8
FR-HIST	Every change to a task recorded with author and time; shown in the task inspector.	2
FR-AI	Weekly summary; meeting minutes; assignment recommendation; goal decomposition; git-diff analysis; workload and overdue statistics; three-tier fallback.	8
FR-NAV, FR-I18N, FR-MKT	Landing page with sign-in at the top right; English/Vietnamese switch for the whole interface; demo video recorded from the real application.	15

2.3.1. Scrum workflow

The Scrum requirements define a small state machine for a sprint:



Figure 2.2. *Sprint lifecycle*

Only a PM may change a sprint; at most one sprint per project is active; a completed sprint accepts no new work; deleting a sprint returns its tasks to the backlog and never deletes work.

2.4. Non-functional requirements

Table 2.4. *Main non-functional requirements*

Quality	**Requirement**
Responsiveness	A local change is rendered immediately; the network call never blocks rendering.
Availability of AI	No AI endpoint returns an error because the model is unavailable; a deterministic answer is served instead and the degradation is logged.
Security	Authorisation is enforced by the server on every project-scoped route; outside development the service refuses to start with a weak secret, demo accounts, wildcard CORS or an out-of-date schema.
Data integrity	Schema changes only through versioned migrations; foreign keys enforced; UTF-8 (utf8mb4) end to end so Vietnamese text is never truncated.
Localisation	Every visible string exists in English and Vietnamese; AI answers follow the interface language.
Testability	Backend and frontend suites pass on a clean checkout and run in CI.

CHAPTER 3: SYSTEM DESIGN

This chapter describes the architecture, the data model, the API, and the design of the three cross-cutting concerns that shaped the system most: authorisation, the AI cascade and the offline policy.

3.1. Overall architecture

Koshi is a client–server system. The browser runs a Vue single-page application that keeps a local copy of the open project; an nginx container serves the application and forwards `/api` to a FastAPI service, which owns the MySQL database and calls the AI provider. For public access, a Cloudflare Tunnel connects outwards from the host to Cloudflare's edge, so no port is opened on the router and TLS is terminated by Cloudflare (Figure 3.1).

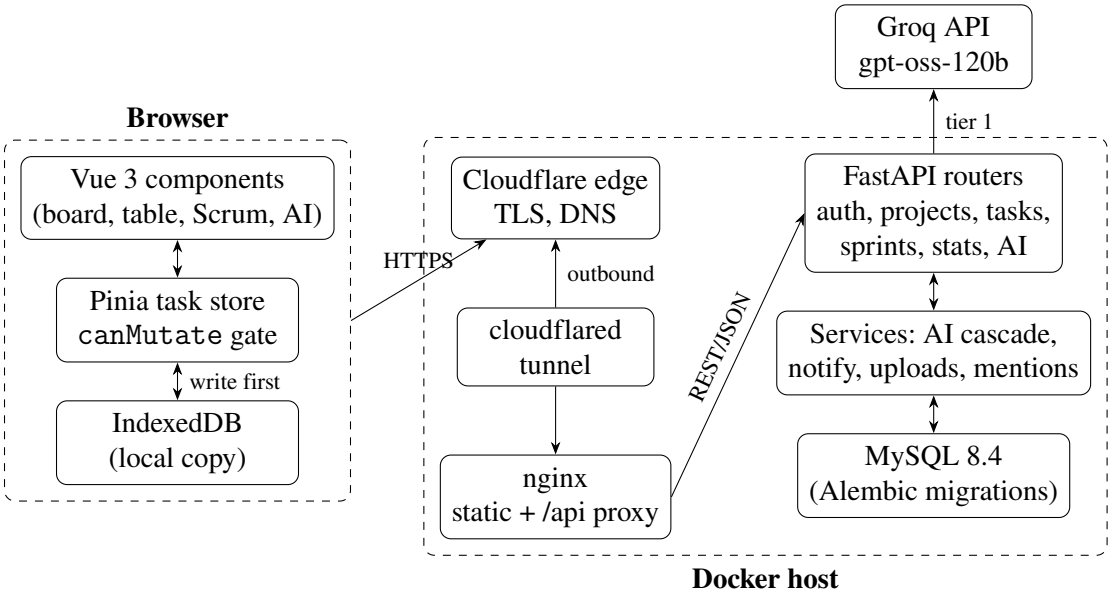


Figure 3.1. *Deployment and component architecture*

The frontend is organised by responsibility: `components/` (26 Vue components), `stores/` (task, theme and language state), `lib/` (pure, framework-free modules: topological sort and critical path, keyboard dispatcher, urgency ordering, git-diff parser, mention parsing, translations), and `services/api.ts`, the only place that talks to the server. The backend follows the same separation: `routers/` (HTTP), `schemas/` (Pydantic contracts), `models/` (SQLAlchemy entities), `services/` (AI, notifications, uploads) and `security.py` (authentication and the project-scope guards).

3.2. Data model

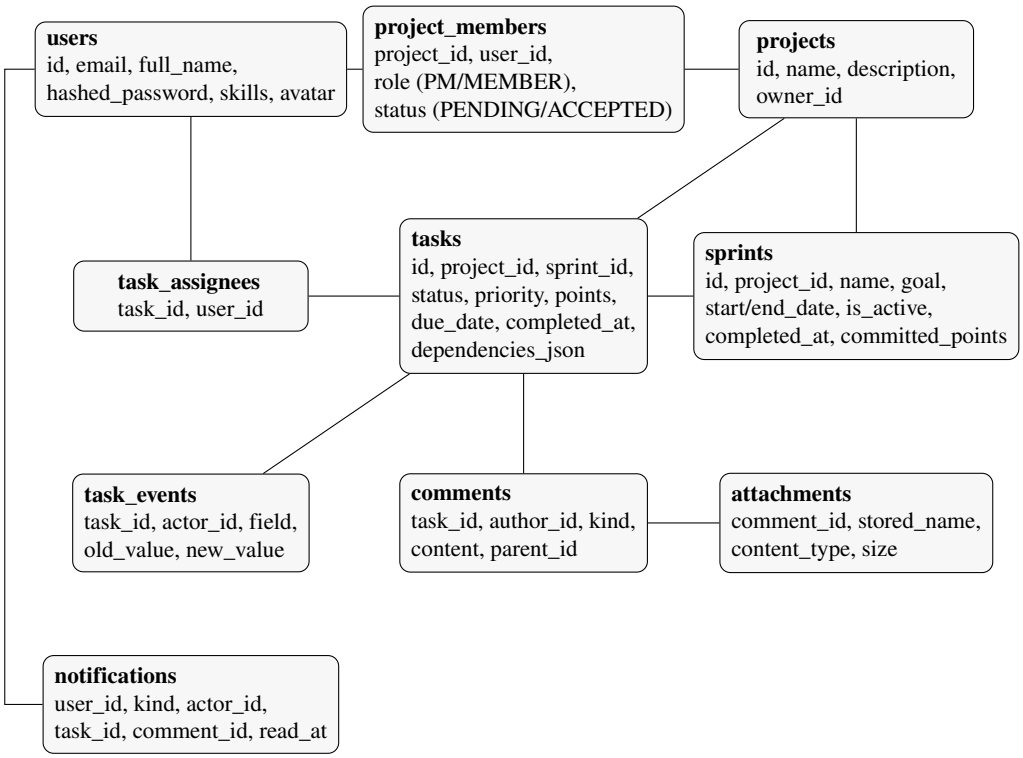


Figure 3.2. *Entities (10 tables)*

Three design decisions matter most:

- **Authority lives on the membership, not on the user.** `users` has no role column; `project_members` holds the role and the invitation status. A PENDING membership grants nothing: the lookup used by every guard returns it only once accepted.
- **Several assignees per task** through `task_assignees`; each must be an accepted member of the task's project.
- **History and Scrum need time.** `tasks.completed_at` is set when a task enters DONE and cleared when it leaves, which is what the burndown reads; the sprint's `committed_points` snapshots the commitment when a sprint starts, which is what velocity compares against. `task_events` stores display strings rather than foreign keys, so an entry still reads correctly after the person or sprint it names is renamed or deleted.

The schema is created and changed only by nine Alembic migrations; outside development the service refuses to start if the database is not at the latest revision.

3.3. API design

The REST API has 51 endpoints under `/api`; Table 3.1 summarises them.

Table 3.1. *API overview*

Router	Endpoints	Main operations
`/auth`	4	register, login, Google sign-in, current user
`/users`	5	list with workload, update own profile, avatar upload/serve/delete
`/projects`	11	my projects, create, delete, members, invite, accept/decline, change role, remove
`/tasks`	11	list/filter, create, update, delete, cycle status, comments, attachments, history
`/sprints`	9	list, create, update, start, complete, delete, statistics, burndown, velocity
`/stats`	2	team workload, overdue tasks
`/notifications`	4	list, unread count, mark one / all read
`/ai`	5	weekly summary, meeting minutes, assignment, decompose, sprint retrospective

Errors follow HTTP semantics: 400 for an invalid reference (an unknown dependency, a sprint of another project), 403 when a member attempts a PM action, 404 for a project the caller cannot see—*not* 403, because revealing that a project exists is itself a leak—and 409 when a second sprint is started while one is active.

3.4. Authorisation design

Every project-scoped route calls one of two guards, `require_member` or `require_project_pm`, before touching data. The user interface hides controls a member cannot use, but this is only an affordance: the server is the security boundary [14]. During the project a code change proposed for the main branch made the client *pretend* to be PM after the server had refused a role change; it was rejected for exactly this reason. Sessions are JWTs signed with a per-deployment secret that is generated on the host and never committed or copied into an image; the startup check refuses weak or default secrets.

3.5. AI cascade

Each AI feature calls one function that tries three tiers in order (Figure 3.3).

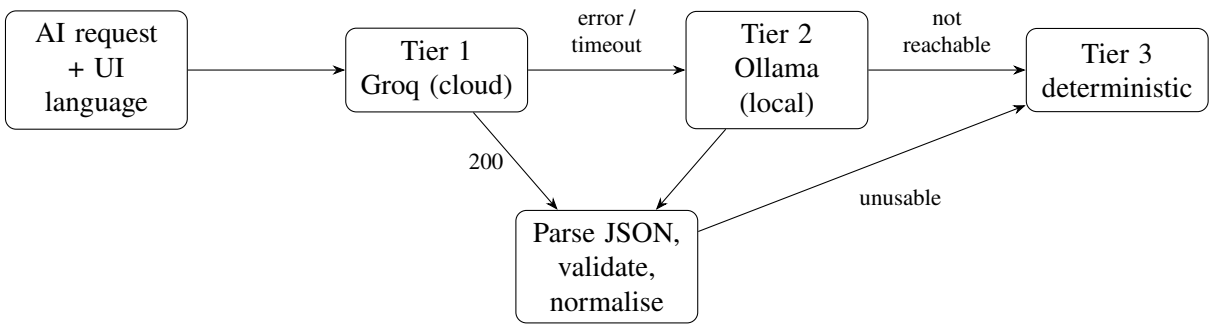


Figure 3.3. *Three-tier AI cascade*

The deterministic tier builds an answer from the project's own data (for example, the retrospective counts committed and completed points, lists blocked tasks with their reasons and detects scope added after the sprint started), so the feature still works with no model at all. Falling through to tier 3 is logged as a degradation; this matters because the tiers return the same shape and an expired key would otherwise look perfectly healthy. The interface language is sent with each request (`Accept-Language`) and added to the prompt, so an English interface receives English answers even when the tasks are written in Vietnamese.

3.6. Offline policy and synchronisation

The store applies every change to the screen and to IndexedDB first, then sends it to the server. A single property, `canMutate`, decides whether a write is allowed: always when online; when offline, only in a personal project. Every control that writes—including the AI tools that insert tasks—checks it itself, because the store refuses silently and a control that did not check would report success having written nothing. Two such defects were found and fixed during testing.

3.7. User interface design

The interface has three screens without a router: the landing page for visitors, the board, and the profile page. Modals provide the task inspector, the Scrum panel and the AI tools. All visible text comes from a translation table with an English and a Vietnamese entry for every key (more than 500 keys); a missing Vietnamese key is a compile error, and an automated test fails if a component contains literal text.

CHAPTER 4: IMPLEMENTATION AND TESTING

This chapter presents the implemented system through screenshots taken from the running application with the interface in English, then the testing strategy and its results, and finally the deployment.

4.1. Development environment

Table 4.1. *Development environment and project size*

Operating system	Ubuntu Linux, Docker Engine with Compose v2
Frontend	Node.js 22, pnpm 10, Vite, Vue 3, TypeScript
Backend	Python 3.12, FastAPI, SQLAlchemy 2, Alembic
Database	MySQL 8.4 (production), SQLite (development, tests)
Source code	about 13,300 lines frontend, 3,400 lines backend (non-blank)
Test code	about 5,200 lines frontend, 2,800 lines backend
Repository	65 commits; 26 Vue components; 51 API endpoints; 10 tables; 9 migrations

4.2. Implemented features

4.2.1. Landing page and authentication

A visitor first sees a marketing page with the features, a demo video and pricing; the sign-in button is at the top right and the language can be switched between English and Vietnamese (Figure 4.1). The demo video is a genuine screen recording: a script drives the real application in a browser and captures it, so it can only show features that work. Registration asks for a name, an email and a password—never a role (Figure 4.2).

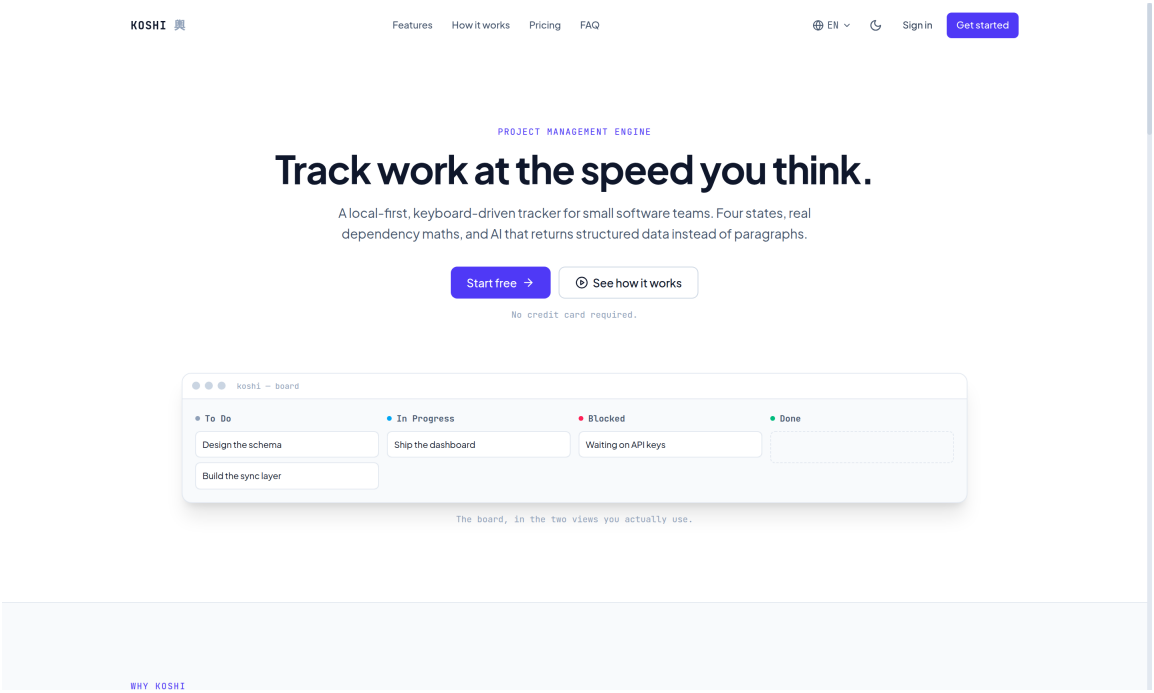


Figure 4.1. *Landing page*

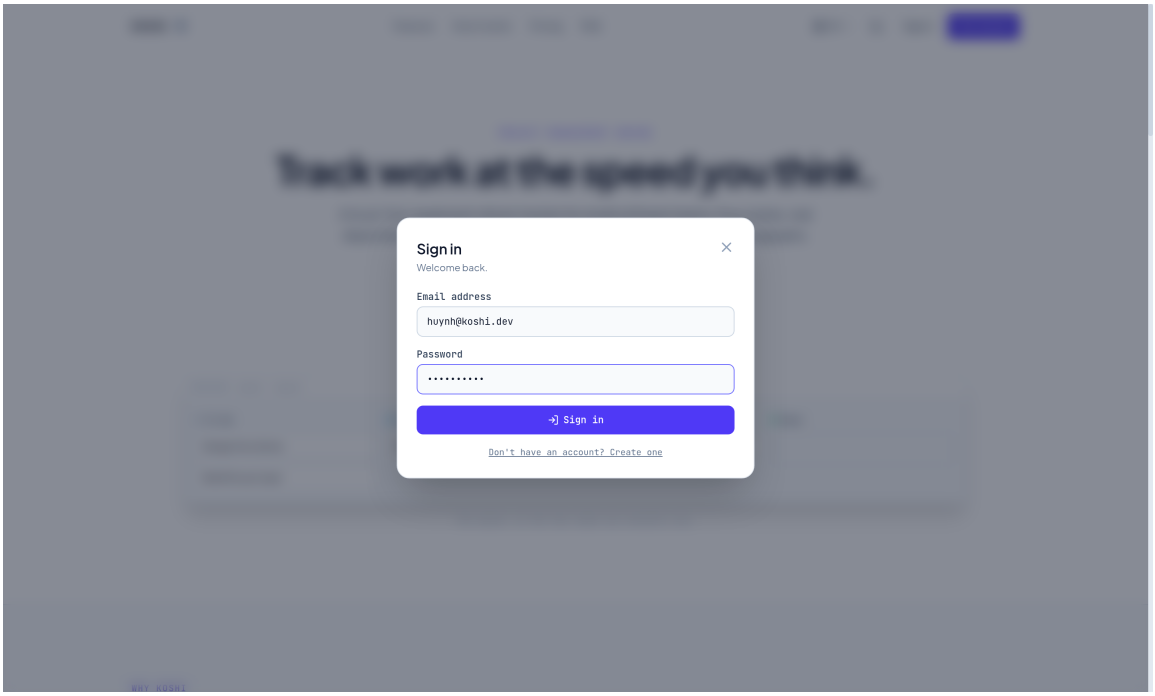


Figure 4.2. *Sign-in dialog*

4.2.2. Projects, members and roles

The dashboard lists the user's projects with the user's role in each, creates projects and manages the team (Figure 4.3). Adding a member by email sends an invitation; the person joins only after accepting it.

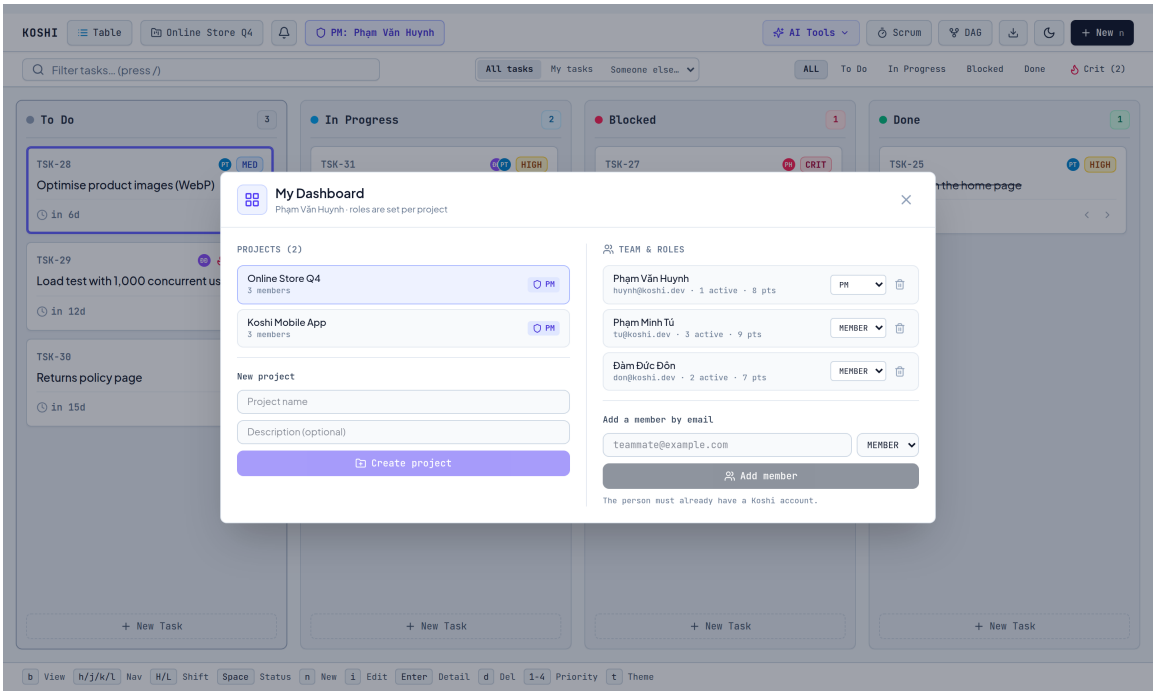


Figure 4.3. *Project dashboard with per-project roles*

4.2.3. Kanban board and table view

The board has four columns. Cards show the key, title, priority, assignees' avatars, the deadline ("in 9d", "3d overdue"), the blocking reason and a flame icon for tasks on the critical path. Ordering is deadline-first: a low-priority task due yesterday ranks above a critical one due next month. The same data is available as a table (Figures 4.4 and 4.5); the key `b` switches between them.

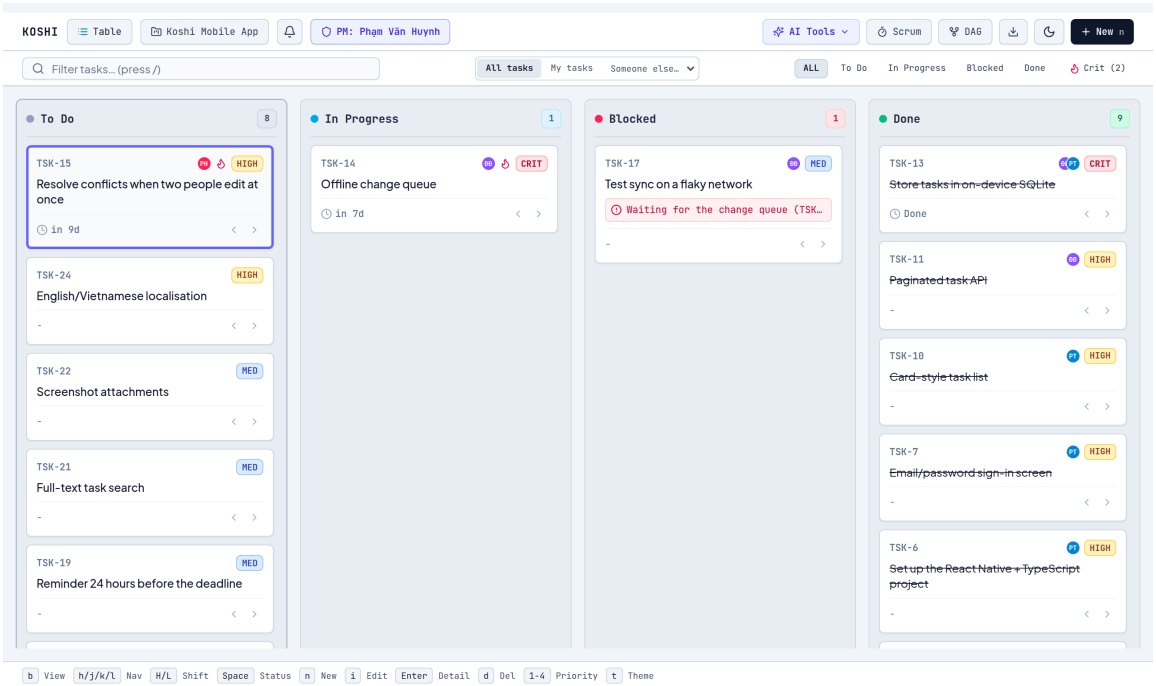


Figure 4.4. *Kanban board*

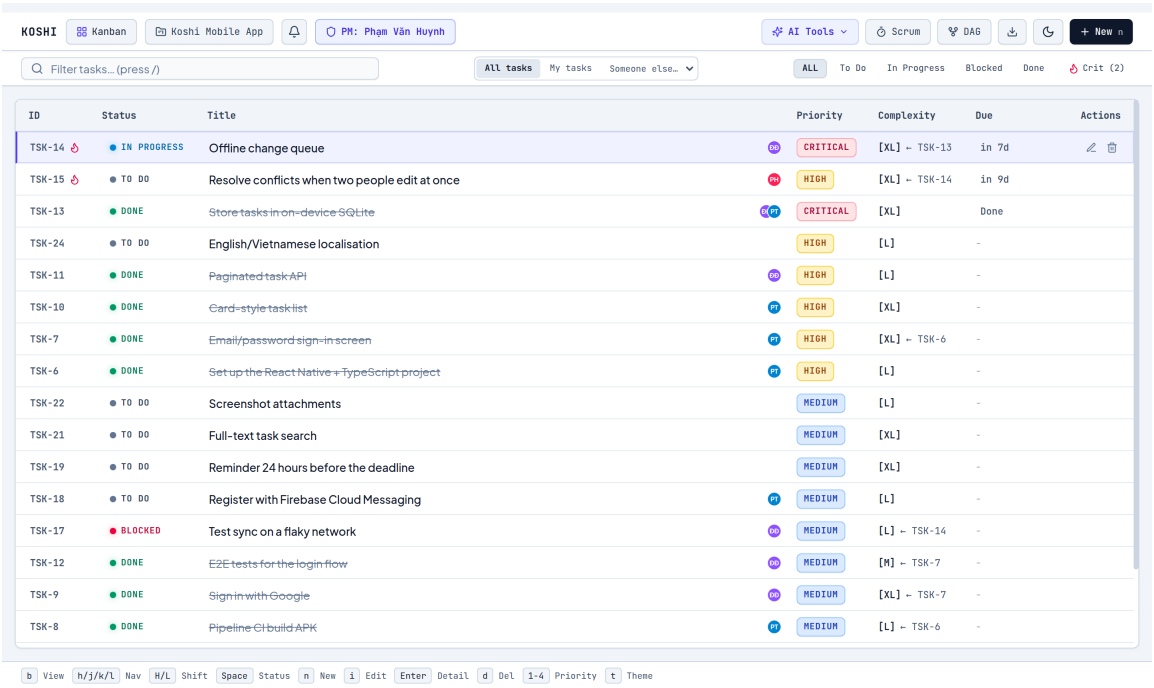


Figure 4.5. *Table view*

Every action has a key (Figure 4.6): $h/j/k/l$ move the selection, Space cycles the status, H/L move a task between columns, 1–4 set the priority, n creates a task and $/$ searches.

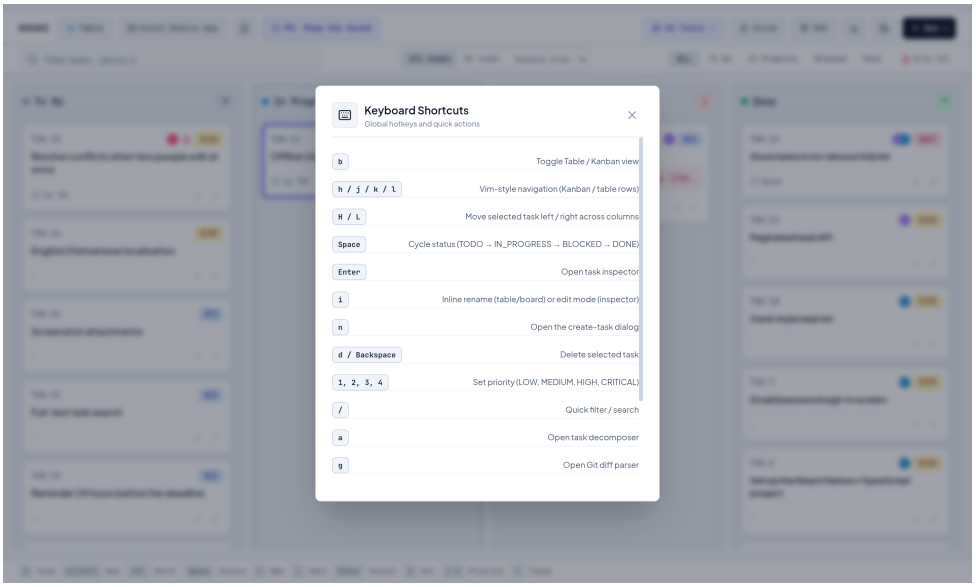


Figure 4.6. *Keyboard shortcuts*

4.2.4. Task inspector: discussion, evidence and history

Opening a task shows its fields, dependencies and acceptance criteria, a discussion thread with @mentions and one-level replies, attachments, and the change history recorded by the server (Figure 4.7). Moving a task to DONE invites the user to attach evidence that the work is finished.

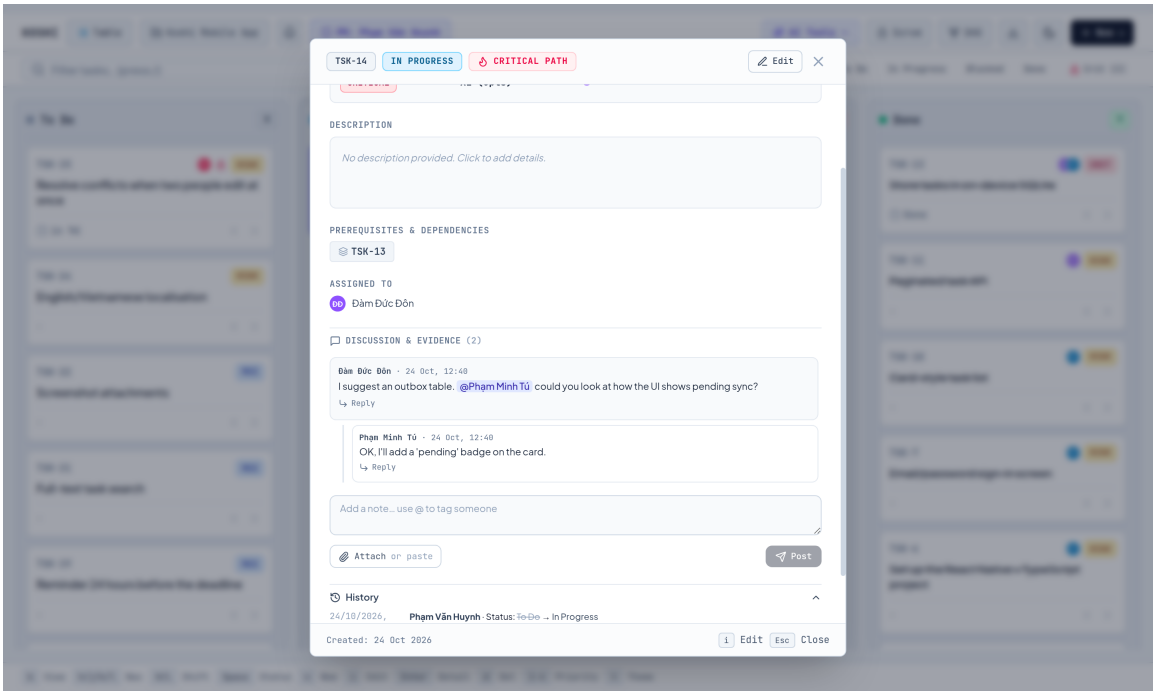


Figure 4.7. *Task inspector with discussion and history*

4.2.5. Scrum

The Scrum panel has five tabs. *Sprints* (Figure 4.8) lists planned, active and completed sprints with their points and commitment; a PM creates, starts and completes them, choosing where unfinished work goes. *Planning* (Figure 4.9) moves items between the product backlog and a sprint. *Burndown* (Figure 4.10) plots remaining points against the ideal line; *Velocity* (Figure 4.11) compares committed and completed points per sprint with the average; *Retrospective* (Figure 4.12) asks the AI for what went well, what to improve and concrete actions, grounded in the sprint's numbers.

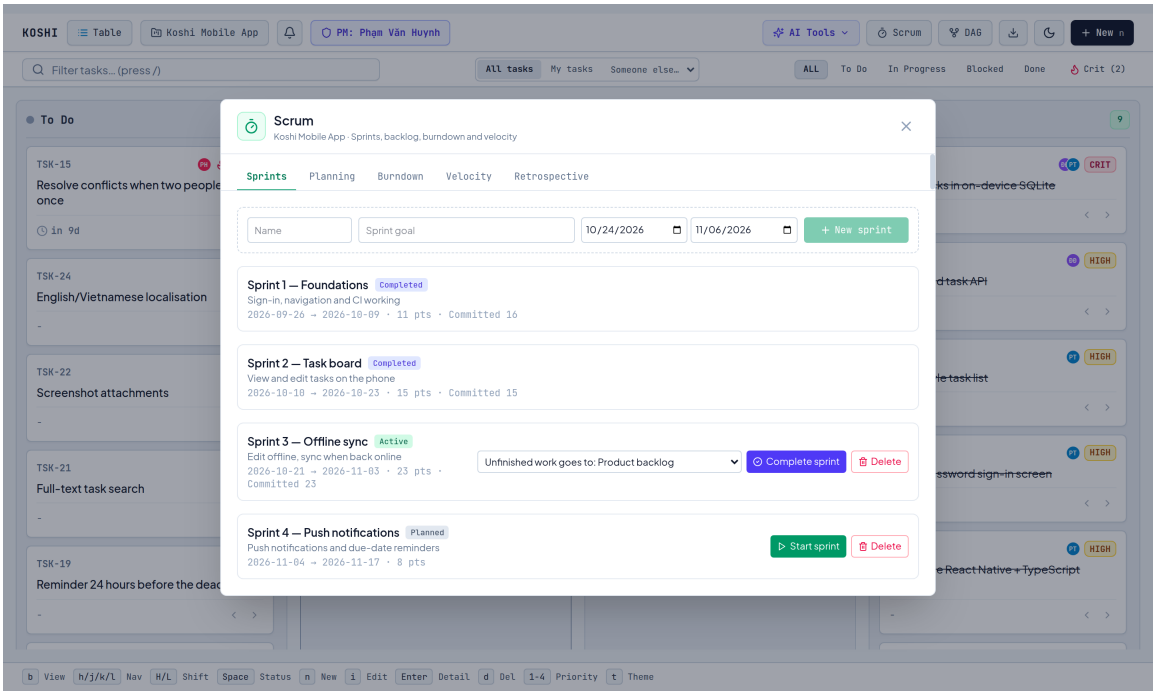


Figure 4.8. *Scrum panel: sprint lifecycle*

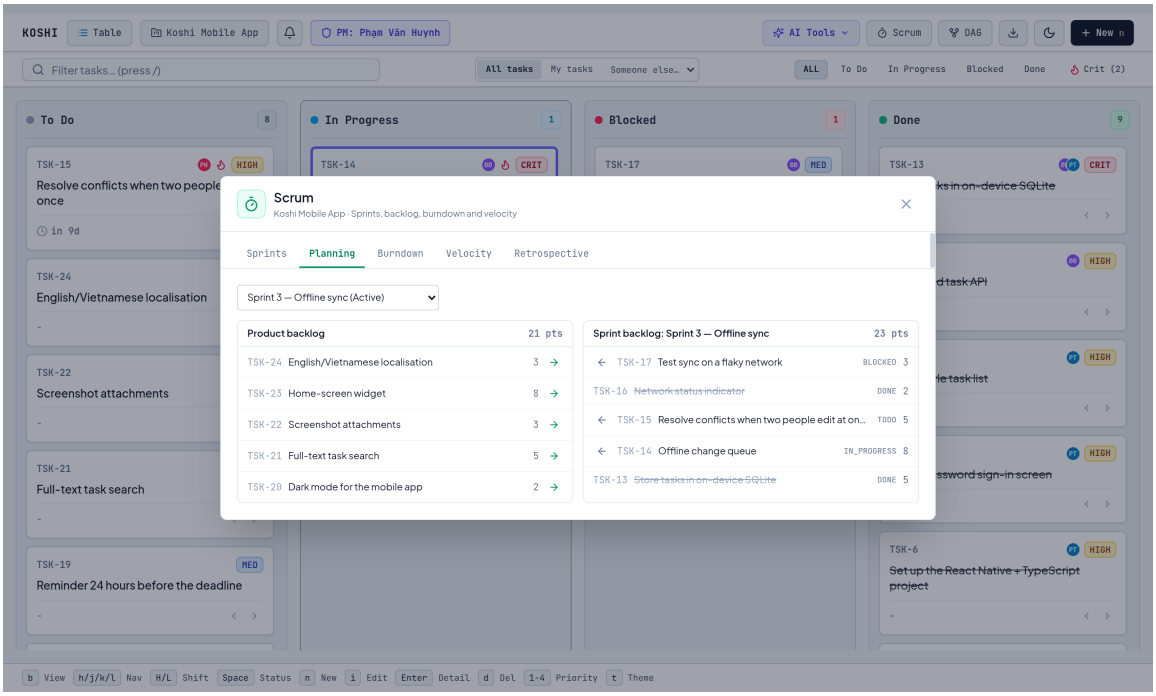


Figure 4.9. *Scrum panel: backlog planning*

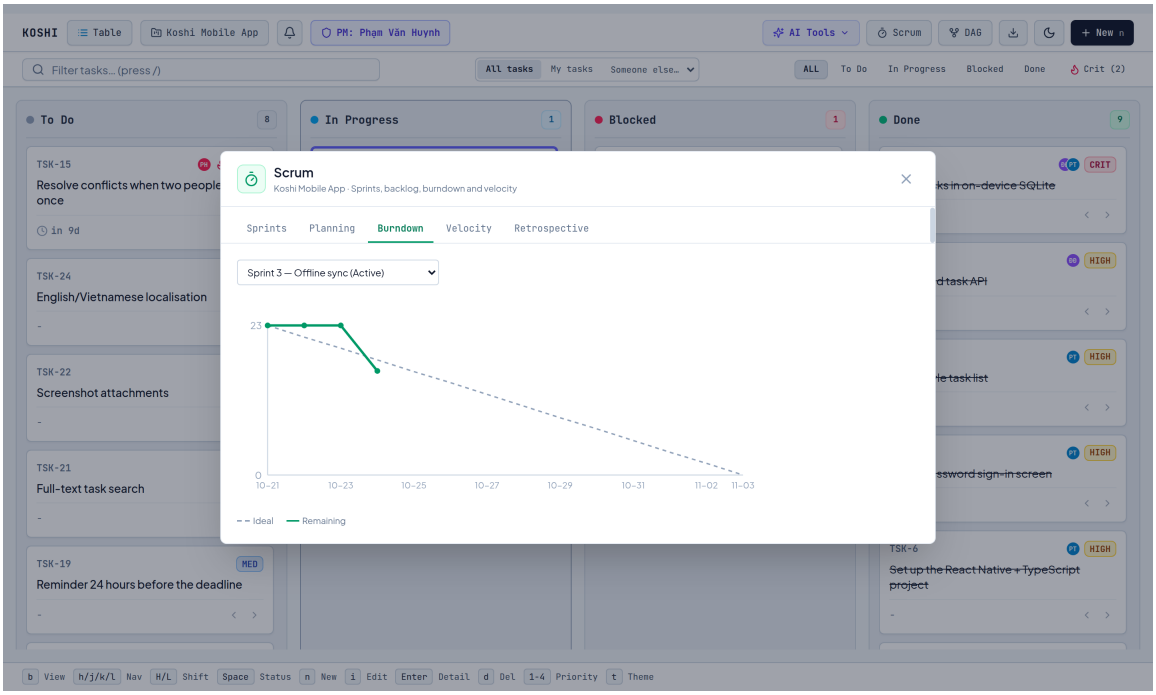


Figure 4.10. *Scrum panel: burndown chart*

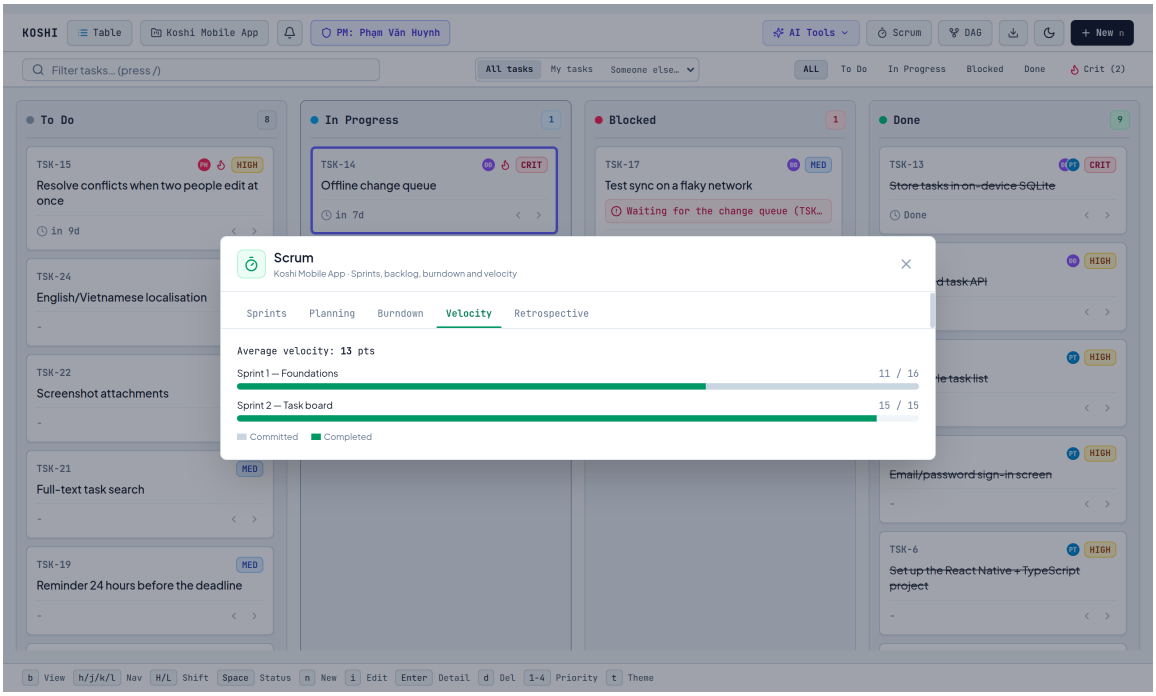


Figure 4.11. *Scrum panel: velocity*

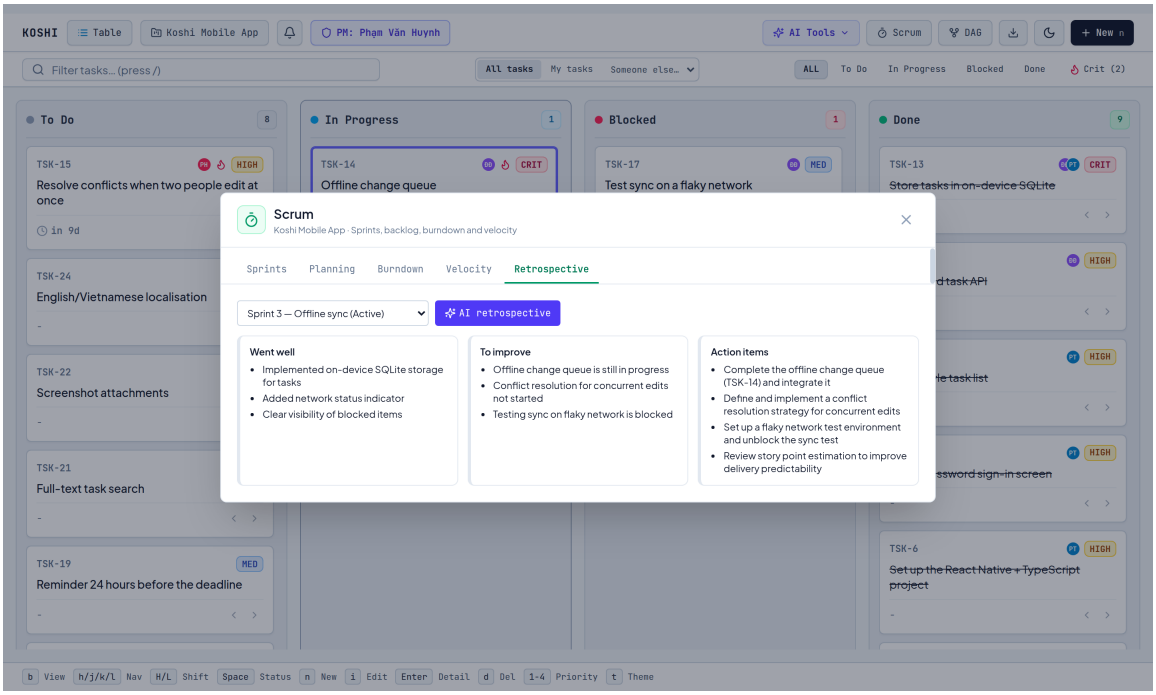


Figure 4.12. *Scrum panel: AI retrospective*

4.2.6. AI tools

Four further AI tools are available from the *AI Tools* menu. The weekly summary reports progress, risks and priorities (Figure 4.13); the meeting-minutes generator turns raw notes into topics, action items with an owner and priority, and decisions (Figure 4.14); smart assignment shows each member's load and skills and recommends an assignee with a rationale and a risk assessment (Figure 4.15); the goal decomposer breaks a goal into dependent subtasks that can be inserted into the board in one step (Figure 4.16).

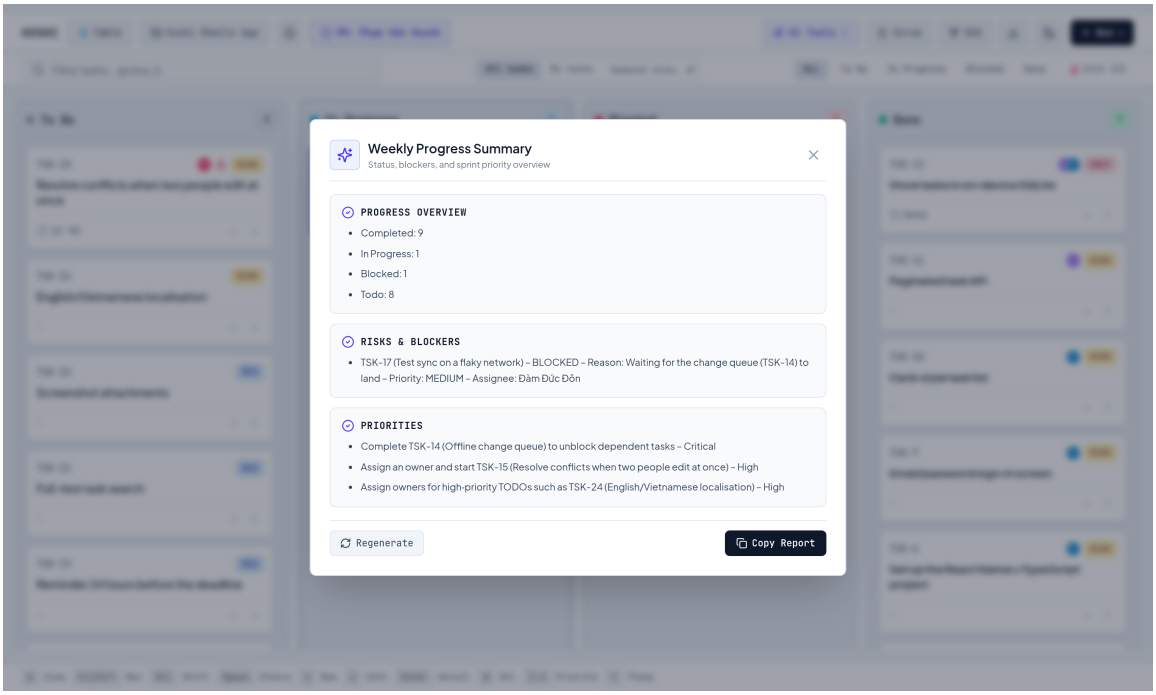


Figure 4.13. *AI weekly progress summary*

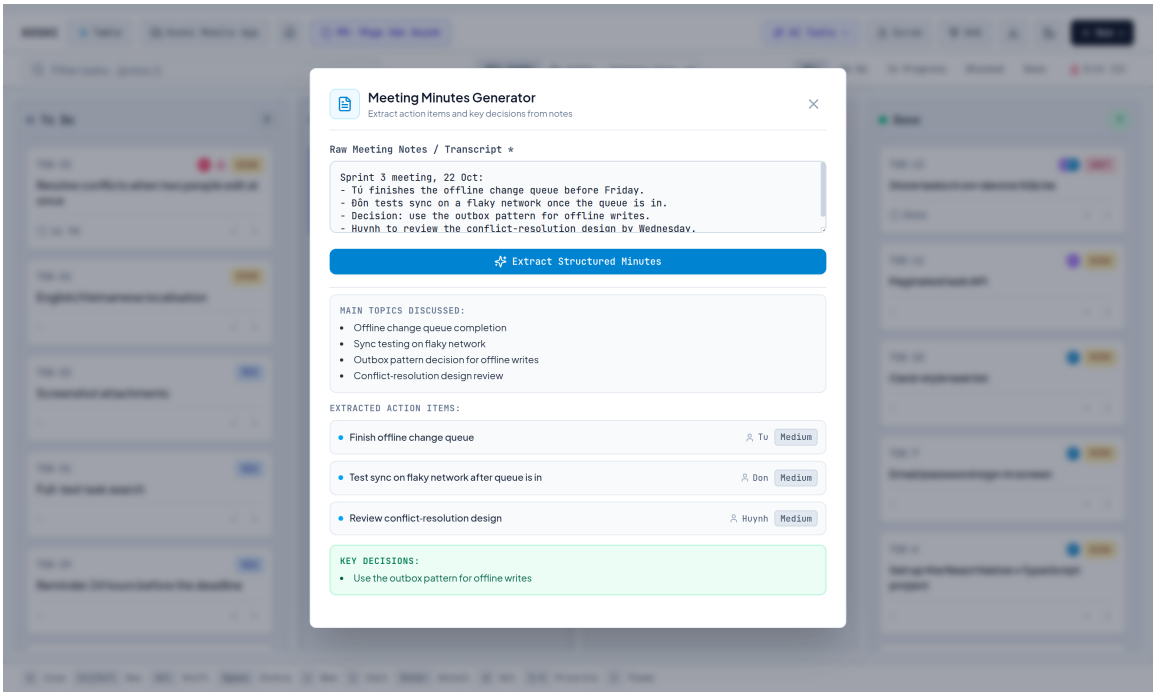


Figure 4.14. *AI meeting minutes*

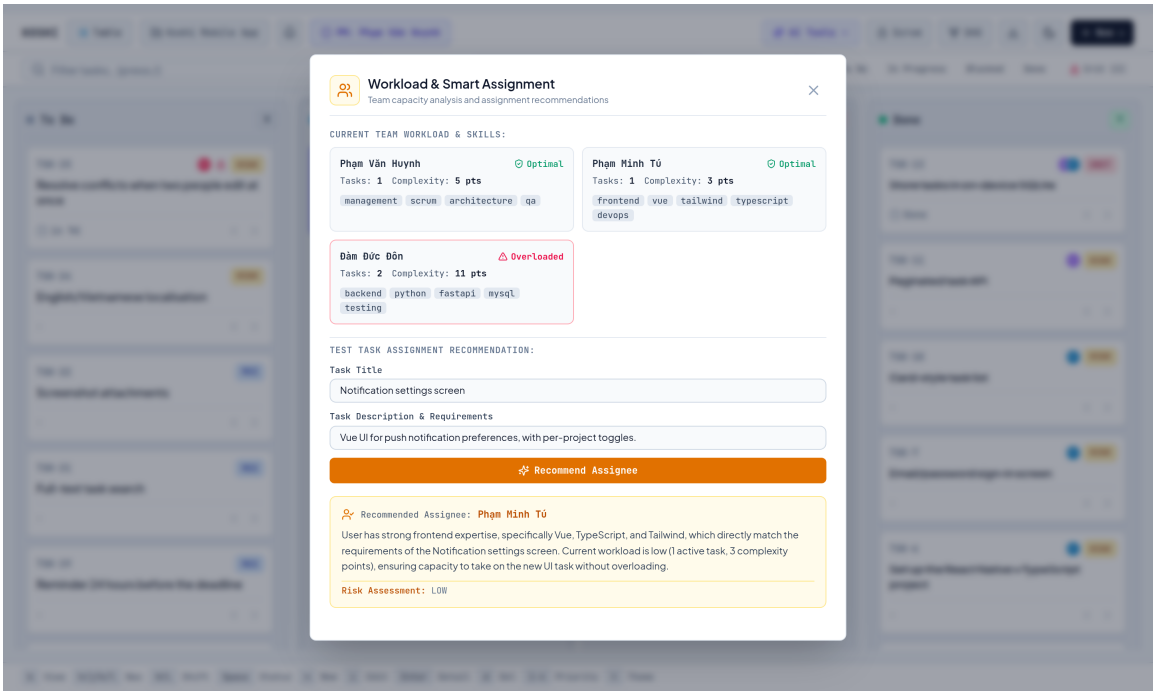


Figure 4.15. *AI workload analysis and assignment*

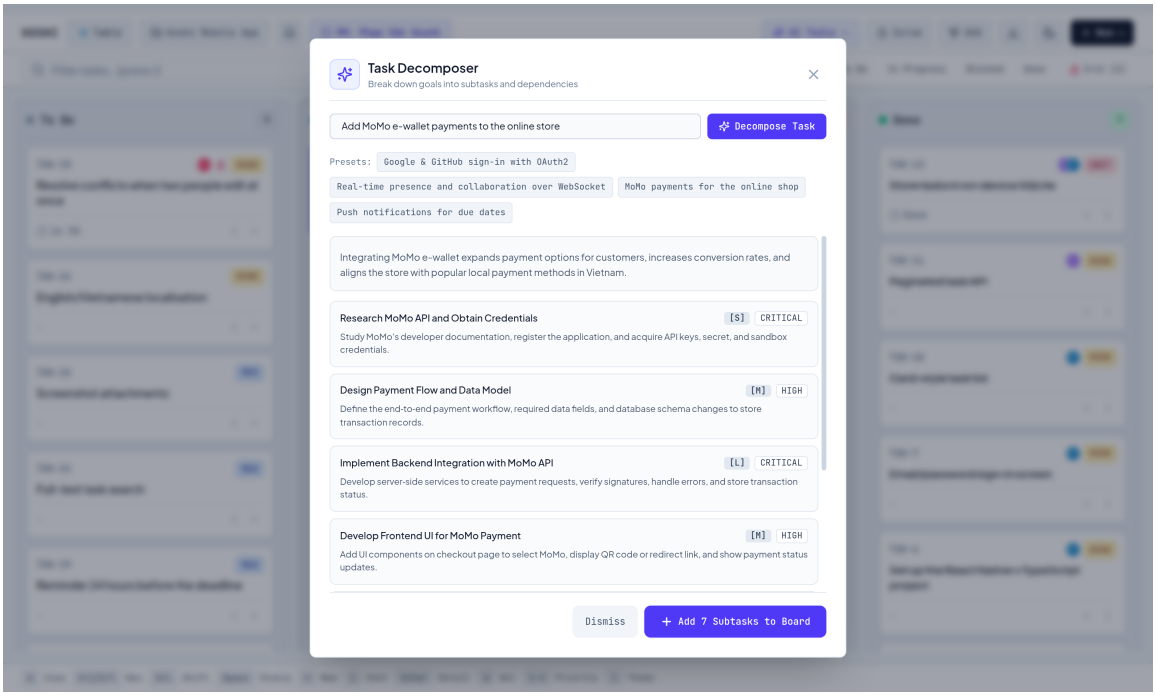


Figure 4.16. *AI goal decomposer*

4.2.7. Dependency graph, notifications and profile

The DAG view lists tasks in topological order and marks the critical path (Figure 4.17). The notification feed reports mentions, replies, assignments and invitations—never the user's own actions (Figure 4.18). The profile page edits the name, skills (used by the assignment recommender) and avatar (Figure 4.19).

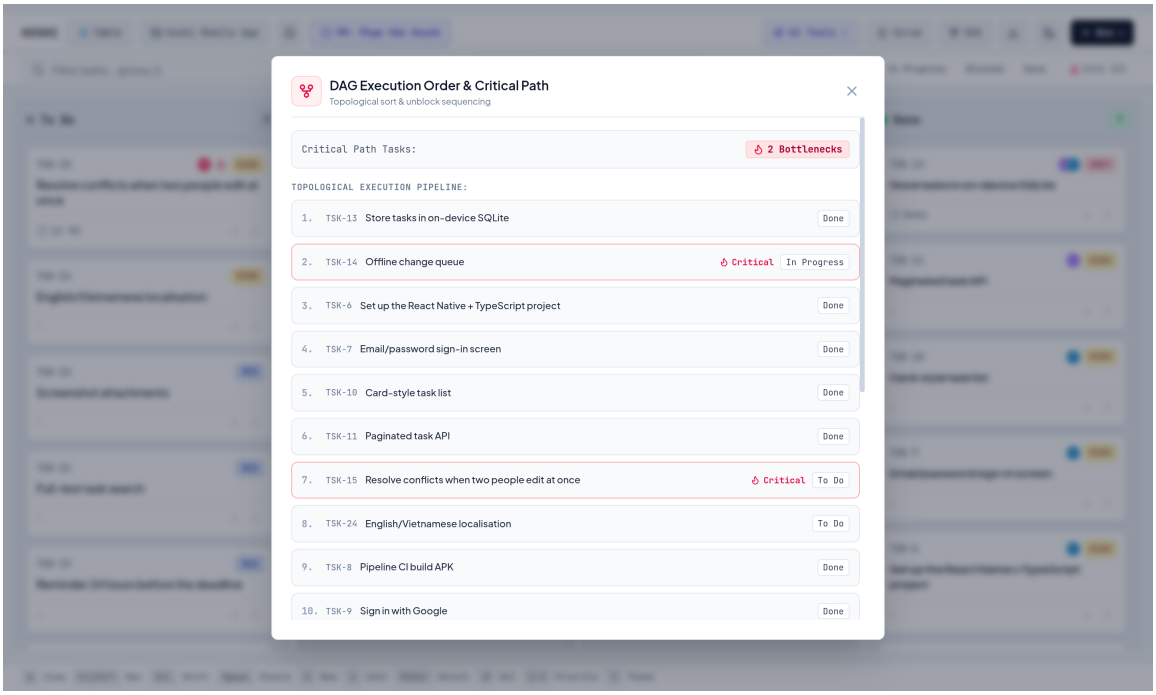


Figure 4.17. *DAG execution order and critical path*

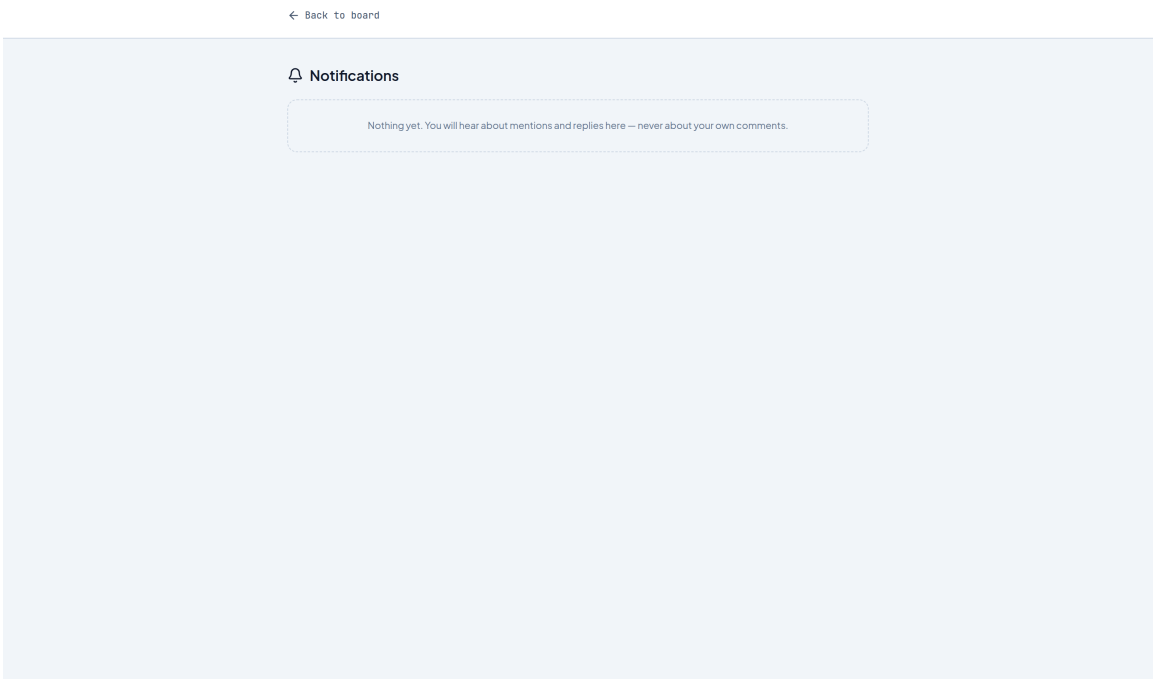


Figure 4.18. *Notification feed*

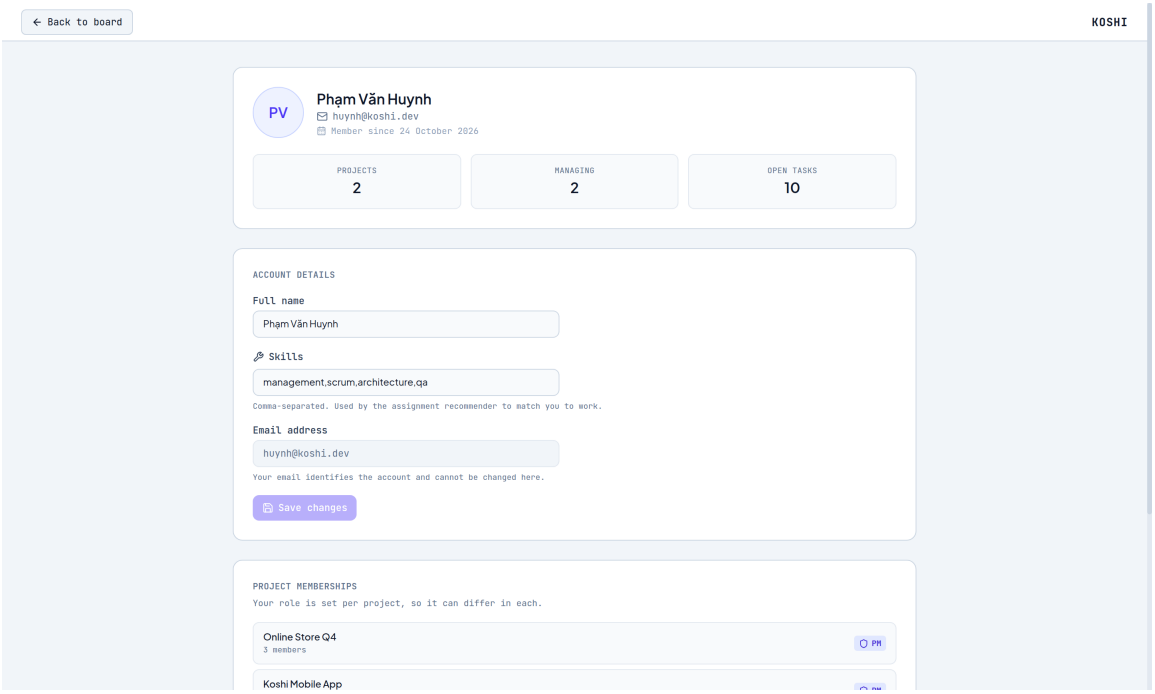


Figure 4.19. *Profile page*

4.3. Testing

4.3.1. Strategy

Testing had four layers:

1. **Backend tests** (pytest, 13 files) exercise the API through a test client: every role and permission rule (including the negative cases—a member receiving 403, an outsider receiving 404), the status cycle, Scrum rules, history, migrations, startup safety checks, and the AI cascade with each tier simulated at the HTTP boundary.
2. **Frontend tests** (Vitest, 19 files) cover the pure algorithms, both stores, the keyboard dispatcher and the components, mounted in a simulated DOM. One test reads every component template and fails if any literal text is not translated.
3. **End-to-end smoke test**: a script seeds a running instance through the public API and performs 32 checks across all features, including the four AI endpoints against the real model.
4. **Browser automation**: Playwright drives the real application to record the demo video and the screenshots in this chapter, and scans each screen for text in the wrong language.

Tests were themselves tested by mutation: a guard was removed or a value changed on purpose, and the change was accepted only if a test failed. For example, removing the check that a sprint belongs to the task's project made two Scrum tests fail; removing the filter on accepted members made the assignment test fail.

4.3.2. Results

Table 4.2. *Test results*

Suite	**Tests**	**Result**
Backend (pytest, SQLite)	177	all passed
Frontend (Vitest)	442	all passed
TypeScript type check and production build	–	passed
End-to-end smoke test on the public instance	32 checks	0 failed
Mixed-language scan of 20 screens (English UI)	20	none found

4.3.3. Defects found

Testing found real defects, each recorded in the development book with its cause. A selection:

Table 4.3. *Selected defects found and fixed*

ID	**Defect**	**How found**
F-32	The published backend image contained the developer's `.env` with the signing secret and a database.	Inspecting the built image
F-37	The documented deploy command copied development settings and databases over production.	Reviewing the command before running it
F-45	All attachments and avatars rendered as broken images: an `<img>` tag cannot send an authorisation header.	Manual testing
F-47	SQLite foreign keys were never enforced, so deleting a task left orphaned rows.	Migration to MySQL
F-48	A task could be planned into another project's sprint.	Writing the Scrum tests
F-49	A finished task was labelled "10d overdue".	Screenshot review
F-50	The "AI" goal decomposer never called a model; it returned a fixed template.	Testing against the real model
F-51	The assignment recommender could suggest someone who had not yet accepted the invitation.	Code review while testing AI

ID	Defect	How found
F-53	A proposed change faked the PM role in the browser after the server refused it.	Review of incoming changes
–	21 of 26 components contained untranslated text, so the two languages were mixed.	Team member's report; fixed and guarded by a test

4.4. Deployment

The production configuration runs four containers: MySQL, the FastAPI backend, nginx with the built frontend, and `cloudflared`. A deployment script builds the images, verifies that no secret or database is inside them, backs up the database, applies migrations in a separate step, starts the services and checks that the instance is healthy, at the latest schema revision and in a production posture (no demo accounts, no wildcard CORS, a non-default secret). The tunnel publishes the instance at `https://koshi.huynhca.com`, where the team tested it with three seeded accounts. Continuous integration on GitHub Actions runs both test suites on every push to the main and development branches.

CONCLUSION

Results

The project delivered a working project management system that meets its objectives:

- Accounts without global roles; projects with per-project PM and Member roles, invitations that must be accepted, and authorisation enforced by the server on every route.
- A keyboard-first Kanban board and table view with deadline-first ordering, several assignees per task, dependencies, a critical path and a DAG view.
- The Scrum process: sprint lifecycle, backlog planning, burndown, velocity and an AI retrospective; plus a change history on every task.
- Five AI features backed by Groq with a three-tier fallback, structured and validated output, and answers in the interface language.
- Local-first behaviour with a safe offline policy; a complete English/Vietnamese interface.
- A production deployment on `https://koshi.huynhca.com`, 177 backend and 442 frontend tests, continuous integration, and eight living documents that trace every requirement to code and tests.

Beyond the features, the most valuable lesson concerned verification. Several of the most serious defects—a secret shipped inside the container image, a deploy command that would have overwritten production with development settings, an "AI" feature that never called a model, a language switch that left half the interface untranslated—were invisible to a test suite that was green. They were found by checking the real artefact: inspecting the built image, reading the command before running it, calling the real model, and scanning every screen. The practice of mutation-testing the tests, and of recording every decision with its reason, came from these experiences.

Limitations

- Shared projects are read-only offline; there is no algorithm for merging concurrent offline edits.
- The burndown is computed from a sprint's current membership, so work moved out of a sprint disappears from its history.
- The deterministic AI fallback answers only in Vietnamese.
- The public instance has no login rate limiting, and the performance targets in the requirements (rendering within one frame, memory use) have not been measured.
- The public instance runs on a single machine and is unavailable when that machine is off.

Future work

- Conflict-free replicated data types (CRDTs) so shared projects can also be edited offline.
- An event log per sprint for exact burndown history, and a daily-scrum view.
- Rate limiting and account lockout; managed hosting with automatic backups.
- Native mobile clients and integration with Git hosting (linking commits and pull requests to tasks automatically).

REFERENCES

[1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, pp. 154–178. DOI: 10.1145/3359591.3359737

[2] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978. DOI: 10.1109/C-M.1978.218136

[3] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.

[4] K. Schwaber and J. Sutherland, *The scrum guide: The definitive guide to scrum: The rules of the game*, https://scrumguides.org/scrum-guide.html, 2020.

[5] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962. DOI: 10.1145/368996.369025

[6] J. E. Kelley and M. R. Walker, "Critical-path planning and scheduling," in *Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference*, 1959, pp. 160–173. DOI: 10.1145/1460299.1460318

[7] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[8] OpenAI, *Gpt-oss-120b & gpt-oss-20b model card*, https://openai.com/index/gpt-oss-model-card/, 2025.

[9] Groq, Inc., *Groqcloud api documentation: Openai compatibility*, https://console.groq.com/docs/openai, Accessed October 2026.

[10] Vue.js Team, *Vue.js documentation*, https://vuejs.org/guide/, Accessed October 2026.

[11] S. Ramírez, *Fastapi documentation*, https://fastapi.tiangolo.com/, Accessed October 2026.

[12] M. B. Jones, J. Bradley, and N. Sakimura, *JSON Web Token (JWT)*, 2015. DOI: 10.17487/RFC7519

[13] Cloudflare, Inc., *Cloudflare tunnel documentation*, https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/, Accessed October 2026.

[14] OWASP Foundation, *Owasp top 10:2021*, https://owasp.org/Top10/, 2021.

[15] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

[16] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill Education, 2020.

[17] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[18] ISO/IEC/IEEE, *Iso/iec/ieee 29148:2018 systems and software engineering – life cycle processes – requirements engineering*, 2018.

LECTURER'S COMMENTS

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

Thái Nguyên, tháng … năm …

Giáo viên hướng dẫn